

CSC 640 Course Project (HW4) & Quality Documentation (HW5)

What you build, how you prove its quality, and how you teach it

How to Use These Slides

Source	What it gives you
These slides	The purpose of the project and how HW2 → HW4 → HW5 connect
HW2	The concepts: requirements, testing, and design
HW4/HW5 on Canvas	The exact requirements: counts, required tools, folders, rubrics, peer review, deadlines

When these slides and Canvas differ, **Canvas wins.**

From the Story to Your Project

In the previous login story, we **specified** quality first, **learned** early, **built** little by little, **designed** modules, and **automated** the software development process.

- We know how to build HQSP.
- It's time to use the knowledge to build an HQSP.

Now you apply the same method to **your project**.

Assignment	Question it answers
HW4 Project	<i>What did you build? (the product)</i>
HW5 Quality Documentation	<i>How do you teach others to build quality software? (using your project as evidence)</i>

Choose Your Project

	Option 1: Your Own Project	Option 2: REST API (default)
Domain	Your company, course, or personal interest	A domain you choose
Stage 1	A working baseline	Raw PHP + MySQL REST API
Stage 2	Approved higher-level tools or automation	All required: Laravel, shell scripts, Docker, Hugo, documentation deployment with GitHub Actions

The domain and technologies may differ, but **both options must produce equivalent evidence of**

Use this course for your resume and portfolio:

- How would you share your experience of building HQSP?
- What would you like to add to your resume?

Choose option 1 or 2 based on your answers to these questions.

Propose Your Own Project (Option 1)

1. Write a **one-page proposal**:

- the **problem domain**,
- your **two stages**,
- your **constraints, deliverables, and "done" criteria**,
- how **each rubric row maps** to an equivalent deliverable and how it is verified.

2. Get the professor's agreement on the mapping before implementation.

- Do not mark rows N/A or redistribute points yourself.

3. Present it briefly in class for your peer reviewers.

The following slides use Option 2 as the example; Option 1 applies the same steps to its own domain.

The Client Request (Option 2)

Problem domain:

"I need a **REST API server** that provides data from my domain in **JSON**, and an **HTML/JavaScript test page** that calls it and displays results. It must be **easy to change** and **easy to find and fix bugs**."

Read [ASE 230 course project](#) to understand details:

- ASE 230 is **Server-Side Programming**.
- All the course project details---ideas, tools, technology, and code---are clearly explained in the GitHub repository.

Clarify Before Building

Applies HW2 Requirements and Use Cases. Before promising anything, **check the important assumptions:**

Ask about	Example
Users	Who calls the API? Students, admins, the front end only?
Data	Which resources? (<code>/students</code> , <code>/courses</code>) Which fields?
Constraints	PHP version, MySQL, hosting, deadline
Acceptance	What must work for the client to say "done"?

The ASE 230 project example uses **student management**.

- For option 2, you may use a similar domain, but you *must* define your own requirements and write your own code; do not resubmit the ASE 230 project.
- The goal is to build a high-quality API server in two steps (low-level first and then high-level).

Define "Done" Before Building

Applies HW2 Software Testing. Write **checkable** criteria first, then build to pass them.

Criterion (specified)	How it is checked (measured → verified)
GET /students returns 200 and a JSON list	Automated acceptance test
GET /students/{id} with a nonexistent ID returns 404 with a JSON error	Automated acceptance test
POST /students without name returns 400	Automated acceptance test
Existing tests still pass after every change	Run acceptance tests; test CI is optional
Storage details stay behind a clear boundary and	Design change evidence

| Build in Two Stages

Inside each stage, repeat: **check assumptions → build one small feature → get feedback → revise.**

1. **Stage 1: Build a working baseline.**
2. **Stage 2: Use higher-level tools or automation to reduce repetitive work.**
3. **Compare** what changed, and **verify** that the required behavior still works.

Same Acceptance Assertions for Both Stages

- Run the **same acceptance assertions** against Stage 1 and Stage 2; normally only environment settings (base URL, credentials) change. Internal unit tests may differ.
- If an approved contract change is needed, explain it and apply the revised assertions to **both** stages.
- Passing them is **evidence that the tested behavior is preserved**; untested behavior is

| Compare the Two Stages

Ask: What did Stage 2 automate for you, and what did it cost?

- Record the answer in the **HW4 comparison memo**.
- Reuse (link) it in HW5 instead of rewriting it.

Apply What You Learn

From HW2	Where you apply it
Requirements / Use Cases	Clarify Before Building
Software Testing	Define "Done", acceptance tests
Design (abstraction, encapsulation, DI, composition, SOLID)	Design change evidence
UML	Optional: to explain structure and responsibilities

HW3 adds JavaScript, TypeScript type safety, and React components. No single tool makes bugs easy to find; **design, tests, and type checks together** do.

Design Change Evidence

Do not just claim "I used SOLID." Show the chain:

**requirement → affected code → unaffected code
→ design principle → regression tests still pass**

Example: *"Keep deleted students for auditing (soft delete)" changed the schema and the SQL in*

MySQLStudentStore; the controllers did not change because they depend only on

StudentStore (DIP); all acceptance tests passed.

A new shared acceptance test checks that a deleted student disappears from GET /students,

Deliver HW4: The Product

Publish on **GitHub** and submit through **Canvas**.

Both options must deliver:

- two runnable stages with executable acceptance tests and results
- the design change evidence and the comparison memo
- documentation that lets a reviewer set up, run, and test the project

The exact files, counts, and technologies are in the

Peer Review

	HW4 (the product)	HW5 (the teaching document)
Reviewers	One Canvas-assigned classmate per stage	Three assigned classmates
Reviewers check	Setup, acceptance tests, one design claim	Accuracy, evidence, and usefulness for a new team member
Score	Informs the instructor's grade; no separate points	Peers score most of HW5; the instructor checks accuracy, progress, and feedback responses

In both, the author responds with a fix or an explanation and verifies any change. Where and when to submit: Canvas.

Deliver HW5: The Teaching Document

- **Publish on GitHub:** a Markdown report and a Marp presentation, each with its PDF.
- **Submit on Canvas:** the HW5 rubric, with links to them.
- **Use your HW4 project as the required example:** show how higher-level tools helped build quality software, and what they cost.

Write for a **future team member**, in your own words:

- What **quality attributes** matter in your project, and why.
- How **process, design, and automation** manage complexity, and their tradeoffs.
- Your **quality criteria** and how you **measured and verified** them, including the **limitations**.
- Your key **design decisions** and the changes they made easier.
- **Lessons learned** from comparing your two stages.

Start early and **revise the document regularly** all semester; progress is judged by dated, substantive revisions, not commit count.

Reviewers check that every quality claim is backed by evidence. Link to the HW4 comparison and results instead of duplicating them.

AI and Your Responsibility

AI may **generate** code, tests, and documents. **You** remain responsible for requirements, review, and verification.

- **Explain** your implementation, tests, and design decisions.
- **Show evidence** (passing tests), not just the AI's claim that it works.
- **Cite** your sources and AI assistance.

Deadlines, points, and detailed checklists: **Canvas**.